实验报告

五子棋：极大极小搜索与 Alpha-Beta 剪枝

江浩　路仕腾　郭展羽

电子科技大学　2026 年 10 月

一 实验目的

1. 用 Python 实现五子棋博弈搜索的两种经典算法：极大极小过程与 Alpha-Beta 过程。
2. 在同一组局面、同一评价函数和同一候选集合上比较两者的访问节点数与耗时。
3. 分析棋盘大小（9、13、15 路）和搜索深度（1 到 3 层）对算法性能的影响。
4. 做一个可以对弈的网页，供小组之间互相体验和评价。

二 实验原理

博弈树

五子棋是确定性、双人、零和、完全信息的博弈。把每个棋盘状态看作一个节点，每一步合法落子看作一条边，就得到一棵博弈树。根节点是当前局面，向下每一层轮换一次执子方。

极大极小过程

轮到己方时选择价值最大的子节点，轮到对方时假设对方会选择对己方最不利的子节点。搜到终局用胜负值，搜到深度上限用评价函数估值，然后逐层回传到根。

$$V(s) = \max_a V(s') \quad \text{（己方走）}$$
$$V(s) = \min_a V(s') \quad \text{（对方走）}$$
$$V(s) = \mathrm{Eval}(s) \quad \text{（到达深度上限）}$$

实现时采用等价的 Negamax 写法：一个节点对当前执子方的价值，等于各子节点对下一执子方价值取负后的最大值，即 $V(s) = \max_a -V(s')$。终局分值大于任何估值，并按层数修正，使程序优先选择更快的胜利。

Alpha-Beta 过程

搜索时维护两个界：α 是己方在当前路径上已经能保证的最低分，β 是对方允许己方得到的最高分。某个节点一旦出现 $\alpha \geq \beta$，它剩下的兄弟分支不可能影响上层的选择，可以直接跳过。

剪枝不改变根节点的值。若每层有 b 个候选、搜索 d 层，极大极小访问约 b^d 个节点；Alpha-Beta 在最坏情况下相同，在走法排序理想时约为 $b^{d/2}$。所以排序越好，剪得越多。

评价函数

沿横、竖、两条斜线扫描每一条线，统计双方的棋型并加权求和，己方得分减去对方得分即为局面估值。权重按威胁程度递减：连五 1,000,000，活四 100,000，冲四 10,000，活三 1,000，眠三与活二 100，眠二 10。

候选落点

只考虑已有棋子周围两格以内的空位，按进攻与防守价值排序后取前 8 个（网页对弈取前 10 个）。若一步就能成五，只搜索胜点；若对方下一步能成五，只搜索堵点。两种算法使用完全相同的候选与排序，因此节点数的差别只来自剪枝。

三 实验内容

规则采用自由五子棋：黑棋先行，双方轮流在交叉点落子，横、竖、斜任一方向连成五子或以上者获胜，无禁手，满盘无人获胜为和棋。

1. **规则引擎**：棋盘合法性检查、四方向胜负判断、和棋判断。
2. **两种搜索**：在同一递归结构里实现极大极小与 Alpha-Beta，统计访问节点、叶子数、剪枝次数和耗时。
3. **对照实验**：3 种棋盘 × 3 个深度 × 3 个局面 × 3 次重复 × 2 种算法，共 162 次测量。
4. **对弈网页**：水墨场景中的 3D 棋盘，可切换算法、深度和棋盘大小，显示每一步的候选分数、主变化线和剪枝记录。

四 实验器材

项目	配置
编程语言	Python 3.12.7（Anaconda），引擎与服务端仅用标准库
基准测试机	Windows 11，Intel Raptor Lake 处理器（24 个逻辑核心），单进程运行
演示服务器	Linux，8 核 CPU，8 GB 内存，Python 3.12.3，systemd 托管
前端	HTML、CSS、JavaScript，Three.js 绘制 3D 棋盘
数据	results/benchmark.csv（逐次记录），results/summary.json（配置与汇总）

五 实验步骤

1. 实现棋盘与规则，用单元测试覆盖四个方向的连五、长连、边界落子、非法棋盘和满盘和棋。
2. 实现共用的候选生成与评价函数，写出 Negamax 形式的极大极小搜索。
3. 在同一递归中加入 α、β 两个界，得到 Alpha-Beta 搜索；用测试确认两者在每个局面上的根值和落子一致。
4. 固定随机种子 20261013 生成局面，关闭时间限制，按棋盘和深度逐组运行两种算法，每组重复 3 次并交替运行顺序，结果写入 CSV。
5. 汇总节点数与耗时，计算节点减少率和加速比，绘制图表。
6. 搭建网页与 HTTP 服务，部署到服务器，供其他小组对弈体验。

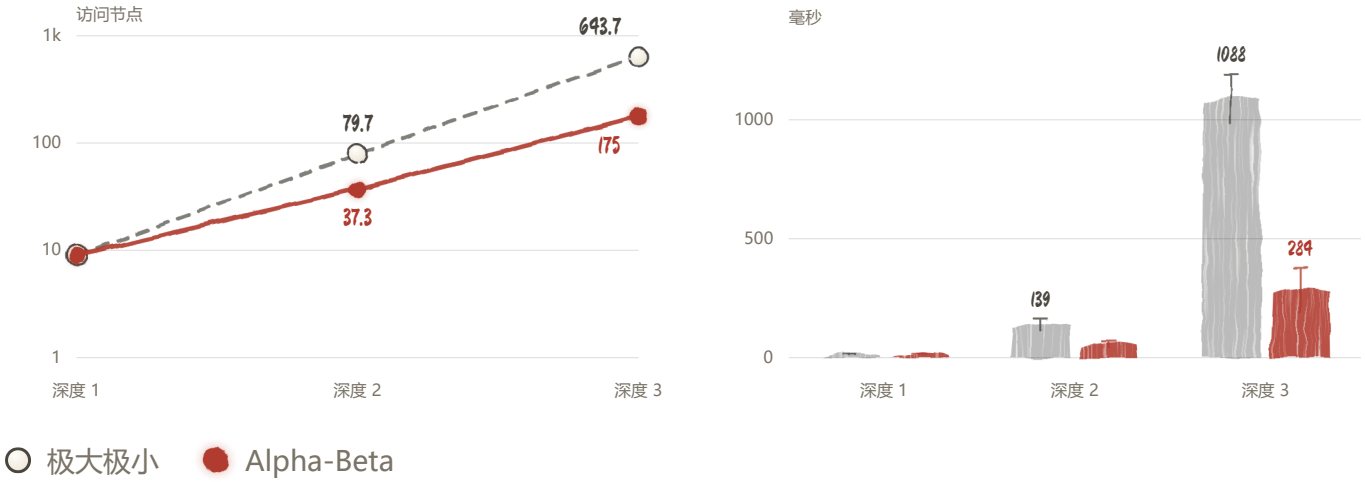
六 实验数据与结果分析

原始数据汇总

棋盘	深度	极大极小节点	Alpha-Beta 节点	减少	极大极小 ms	Alpha-Beta ms
9 × 9	1	9.0	9.0	0%	6.2	6.2
9 × 9	2	79.7	31.7	60.3%	51.1	20.7
9 × 9	3	643.7	171.7	**73.3%**	424.6	126.2
13 × 13	1	9.0	9.0	0%	13.2	12.4
13 × 13	2	79.7	37.3	53.1%	110.5	51.1
13 × 13	3	643.7	175.0	**72.8%**	896.9	238.5
15 × 15	1	9.0	9.0	0%	15.8	15.7
15 × 15	2	79.7	37.3	53.1%	139.5	58.7
15 × 15	3	643.7	175.0	**72.8%**	1087.7	283.9

每行为 3 个局面各测 3 次的均值。节点数包含迭代加深各轮。

节点数与耗时随深度的变化



剪枝的收益随深度增大

深度 1 时只需给根节点的候选打分，没有可剪的分支，两者节点数相同。深度 2 时减少 53% 到 60%，深度 3 时减少约 73%，全部 162 次测量合计减少 **70.2%**。15 路棋盘、深度 3 下，耗时从 1087.7 ms 降到 283.9 ms，快 3.8 倍。

深度每加一层，极大极小的节点数乘 8

极大极小的节点数从 9 到 79.7 再到 643.7，每层约乘 8，正好是候选上限，和 b^d 的理论增长一致。Alpha-Beta 每层约乘 4.4，介于最坏情况 8 与理想排序的 $\sqrt{8} \approx 2.8$ 之间，说明现有排序还有改进空间，这也是创新部分要做的事。

棋盘大小影响单个节点的成本，不影响节点数

候选上限固定为 8，三种棋盘的搜索树形状相同，节点数一致。耗时随棋盘增大：深度 3 时极大极小从 9 路的 424.6 ms 增加到 15 路的 1087.7 ms，约 2.6 倍。原因是评价函数要扫描整个棋盘的所有线，15 路棋盘的线比 9 路多得多。若取消候选上限，棋盘越大空位越多，分支数也会随之增长。

正确性验证

81 组配对搜索（27 个局面与深度组合，各重复 3 次）中，两种算法的根节点值和选择的落子全部一致。引擎 9 项单元测试、服务端 5 项接口测试全部通过，覆盖胜负判断、非法输入、一步必胜、必须防守、超时恢复和满盘和棋。

基线引擎的棋力测试

网页对弈限定每步 2 秒。基线引擎每秒约搜索 800 个节点，在这个时间内能完成 4 到 5 层。我们从它的自我对弈中截取了 10 个存在连续冲四必胜（7 到 17 手）的局面，它只找到 1 个。另外让它下了 16 盘测试对局（每步 1 秒，交换先后手），361 手中有 53 手（15%）落下之后对手立即拥有连续冲四必胜，它输掉的 8 盘每一盘都出现过这种失误。

结论是：固定 2 到 3 层的 Alpha-Beta 能正确应对一步之内的威胁，但看不见更长的强制序列。提高搜索效率、加入威胁空间搜索，是下一步的重点。

七 创新与拓展

三项工作正在进行，完成后在此补充数据。

博弈树加深

增量评估（落子只重算经过该点的 4 条线）、Zobrist 置换表、杀手着法与历史启发排序、迭代加深复用上一轮主变化线，以及 VCF 连续冲四搜索。目标是在同样 2 秒内多搜几层，并找到 10 步以上的强制胜。评估指标：同时间可达深度、有效分支因子、连续冲四题集的解题率、与基线的对弈胜率。

自我对弈强化学习

用 TDLeaf(λ) 训练评价函数的棋型权重：AI 与自己对弈，每一步以搜索得到的主变化线叶子作为学习对象，用相邻两步估值之差更新权重。先在 9 路棋盘上训练，再迁移到 15 路。评估指标：训练过程中对手工权重的胜率曲线、权重变化。

人类反馈

当 AI 前两个候选分数接近时，在网页上请玩家二选一。用 Bradley-Terry 模型拟合这些选择，得到对评价函数权重的修正，并用正则项约束修正幅度，避免棋力下降。评估指标：对留出偏好的预测准确率、对人类落子的预测准确率、修正前后的对弈胜率。

八 实验结论

1. 两种算法在同一候选树上给出相同的根值和落子，Alpha-Beta 的剪枝不改变决策。
2. Alpha-Beta 的收益随深度增大：深度 3 时节点减少约 73%，耗时快 3.8 倍；全部测量合计减少 70.2%。
3. 搜索深度决定树的规模：极大极小每层节点数约乘 8，Alpha-Beta 约乘 4.4。
4. 候选数固定时，棋盘大小不改变节点数，但让每个节点的评估变慢，15 路比 9 路慢约 2.6 倍。
5. 2 秒内 4 到 5 层的搜索不足以发现长的连续冲四，需要更高效的搜索和专门的威胁搜索。

九 总结与心得

这次实验里最花时间的不是写递归，而是让对比公平：两种算法必须共用同一个评价函数、同一组候选和同一种排序，节点数的差别才能归到剪枝上。固定种子、交替运行顺序、保存

逐次原始数据，让每一个数字都能重新算出来。

理论复杂度解释了增长趋势，但真实耗时还取决于每个节点做多少工作。棋盘大小的实验就是例子：节点数完全相同，耗时差了 2.6 倍。

组员各自的心得体会：（待补充）

十 小组互评

其他小组可以在 **对弈页面** 与 AI 下棋，切换极大极小与 Alpha-Beta、调整深度和棋盘大小，并在对局中对 AI 的落子给出反馈。互评结果在体验后补充。

参考资料

1. 课程材料《实验一 五子棋》，电子科技大学。
2. Russell S, Norvig P. Artificial Intelligence: A Modern Approach（第 4 版），第 5 章 对抗搜索。
3. Knuth D E, Moore R W. An Analysis of Alpha-Beta Pruning. Artificial Intelligence, 1975.
4. Baxter J, Tridgell A, Weaver L. TDLeaf(λ): Combining Temporal Difference Learning with Game-Tree Search. 1998.
5. Christiano P 等. Deep Reinforcement Learning from Human Preferences. NeurIPS, 2017.
6. 棋型评分表改编自 husus/gomokuAI-py（MIT 许可）。